ProjectPilot AI Iteration 3 Written Deliverable

CSE 3311 Object Oriented Software Engineering
Team 5
November 1 2026

Iteration outcome

We extended ProjectPilot AI from live manager planning to live specialist deliverables, AI review, and feedback that rewrites affected work. Students can approve a dependency ordered plan, run specialists, review findings, request changes, and export approved deliverables. The application is now available on Vercel [1].

The most important change is that Live API mode no longer uses prepared specialist responses. Each specialist receives the approved requirements, its task contract, and completed prerequisite outputs. Revisions receive the previous deliverable and the requested change. Every new or revised output still requires human approval.

Access and reproducibility

Live application: https://projectpilot-ai-cse3311.vercel.app
Repository: https://github.com/manit0700/projectpilot-ai-cse3311
Local source folder: /Users/manitdankhara/projectpilot-ai

Submission version: Git commit e1af1d5 with peer-review fixes, deployed to Vercel production, deployment dpl_ADbJazYTAK2BwdXr3P4teNAjRZk9. The implementation and regression tests are pushed to the shared master branch. The accompanying submission documents and new recording are separate artifacts.

Current product boundary

Iteration 3 produces structured deliverable text, including code samples when the task requests implementation. It does not assemble a runnable project directory, execute generated code, run generated tests, import a repository, or deploy a generated application. The next iteration will target a complete runnable project for a narrow supported stack. Hosting ProjectPilot on Vercel is distinct from deploying a project generated by ProjectPilot.

Verification summary

All 21 automated workflow tests passed. The production build passed. Hosted browser checks passed for approvals, Markdown download, refresh recovery and mobile overflow. Real hosted API checks passed for specialist generation, review and changed feedback revision. These checks establish application behavior, not the correctness of all generated code or measured usefulness for students [2].

Customers Users and Feedback

Our intended users are individual students building small class, hackathon or portfolio web applications. A first useful project has a few screens, explicit inputs and outputs, and a limited data model. Examples include a study-session tracker, a local task tracker and a club signup prototype. Users need a task plan they can inspect, concrete deliverables, understandable error recovery, and the ability to revise work when requirements change.

No paying customer has been identified. The course professor and peer review team provide evaluation feedback. Their feedback informs development, but it does not substitute for interviews with the intended student users. We have not established retention, willingness to pay or a productivity improvement.

Documented interactions

Source and date	Observation or request	Implementation response
Christoph Csallner, Oct 26, 2026 [3]	Next iteration should add specialists with retry.	Live specialist endpoint, one automatic retry and manual per-task retry.
Professor comments [3]	How can external feedback update a plan or output?	Feedback rewrites selected outputs and their transitive dependents. Editing requirements reopens the task plan.
Professor comments [3]	Demo font is small and planned screens are missing.	Increased shared small-text size. Current screen screenshots and exceptional transitions appear in this report.
Professor comments [3]	Try the app on this course project and narrow example scope.	Real API self-check targets ProjectPilot retry-screen design. Recorded demo targets a small study tracker.
Professor comments [3]	Why multiple agents instead of one agent or an existing coding assistant?	Role contracts and shared prerequisite outputs are explicit. Comparative benefit remains unmeasured.
Team 6 inception review, retained notes [4]	Define scheduling, failure handling, supported tasks and evaluation.	Dependency checks, visible failures, scope examples and developer verification address these design concerns.

Student feedback still needed

No completed external student interview log is included in this iteration. We should test with 3 to 5 students using the same short brief and document consent, date, task, observations, direct feedback and resulting action. Ask them to find a failed task, request a revision, identify what changed and export the result. Record task completion, confusion and a usefulness rating. Do not report planned interviews as completed work.

Proposed next measurement: compare one study-tracker task using ProjectPilot and one using a single-agent workflow, counterbalance the order, and record time, accepted deliverables and interface inconsistencies. The return-use hypothesis is that saved task state and focused revision history reduce coordination effort. That hypothesis needs evidence.

Iteration 2 Peer Review Disposition

Team 6 reviewed Team 5 and supplied 18 findings. The team provided the review for this submission on November 1 [11]. Reviewers are David, Ujjwal Sood and Hardik Kari. They are reviewers, not the authors of this Team 5 deliverable. The following status describes current implementation and submission evidence. Open findings remain release work.

No and finding	Status	Disposition
1 Usefulness slider explanation	Addressed	Usefulness now explains its 1 to 10 scale, persistence/export behavior and lack of effect on agent decisions.
2 Bottom navigation controls	Addressed	Previous and Next controls appear at the bottom of every workflow screen and retain approval gates.
3 Metrics visibility	Addressed	Planning metrics now appear above the main workflow.
4 Input minimums	Addressed	The idea, answers, requirements, task descriptions and feedback enforce minimum detail. Titles and target-user fields have appropriate shorter minimums.
5 Readable report screenshots	Addressed	Current screenshots occupy separate report pages and readable slide crops.
6 Real API verification	Addressed	The hosted demo records real manager, specialist, review and revision responses. Separate provider verification also passed.
7 Target student sessions	Open	No completed 3 to 5 student usability sessions are claimed.
8 Conflicts block final approval	Addressed	Conflicts now block individual output approval and final export. A revision and clean review must resolve them.
9 Rewrite feedback output	Addressed	Live API regenerates actual outputs and dependents, then reviews again. Demo replaces its sample deliverable instead of appending the request.

Peer Review Remaining Findings

No and finding	Status	Disposition
10 Correct save message	Addressed	Final screen correctly states that progress is saved in this browser.
11 AI keyword false match	Addressed	Word-boundary matching prevents availability from triggering an AI question.
12 Keyboard-only workflow test	Addressed	Keyboard-only planning, editing, task board navigation, review, revision, export and reset cancellation passed in Chrome and WebKit.
13 Additional browser suite	Addressed	Critical workflow checks passed in Chrome and the WebKit browser engine, including refresh recovery and export.
14 Live data consent	Addressed	Live access requires explicit consent after a notice listing the data sent to OpenAI.
15 Delayed-request timeout test	Addressed	A delayed provider request abort test checks preserved draft, restore and successful retry. The deadline is scaled to 30 ms for the test.
16 Damaged or old browser saves	Addressed	Invalid JSON, old envelope versions and malformed saved collections keep a recovery copy and a usable new session.
17 Public API usage protection	Addressed	Production manager and specialist calls require an access session signed by the server. The access code stays private and cookies are HttpOnly. There is no claim of per-user distributed quotas.
18 Detailed export regression cases	Addressed	Regression tests check edited requirements/tasks, dependency order, approval, rewritten feedback and special Markdown characters. Two browser engines also check downloaded content.

Priority of remaining review work

The software and testing fixes address 17 of the 18 numbered findings. Finding 7 remains open because genuine student sessions need actual participants. Production access uses a private reviewer code and explicit consent. Keyboard and cross-browser evidence is from Chrome and WebKit, not a claim of a full WCAG audit or manual Safari-app certification.

Updated Project Plan

Period	Status and user value	Evidence or exit condition
Iteration 1	Completed mock workflow and approvals.	Idea, clarification, requirements, task board, review, feedback and export.
Iteration 2	Completed live manager and browser persistence.	Live questions, requirements and tasks, dependency validation and refresh recovery.
Iteration 3 Nov 1	Implemented live specialists, review, revisions and hosting.	21 tests, production build, hosted browser checks and real API self-checks [2].
Next iteration milestone A Nov 2 to 3	Proposed file-based generation for one stack.	Generate a manifest and complete React/TypeScript starter files with stable paths.
Next iteration milestone B Nov 4 to 5	Proposed integration and isolated verification.	Install dependencies, build and run tests with visible logs and bounded repair attempts.
Next iteration milestone C Nov 6 to 8	Proposed preview, ZIP and student evaluation.	A study tracker can run from the downloaded package. Record external feedback before widening scope.

Dates after November 1 are proposed internal targets, not confirmed course deadlines or team commitments. Individual ownership still needs team agreement. Responsibilities are grouped by manager workflow, specialist integration, verification and user research.

Why this order reduces risk

A file manifest and a narrow stack resolve interface and packaging uncertainty before we add more project types. A build check tests integration before a preview can imply success. A ZIP and README make the outcome inspectable outside our app. User evaluation should occur before adding repository import or broader autonomy.

Next iteration success condition

A student enters a small supported web-app idea, approves the plan and receives actual project files, a working preview and a downloadable package. The system reports actual build and test results, including failures. Feedback changes those files and triggers another verification pass. A plan or disconnected code snippets alone do not satisfy this goal.

Deferred scope

Automatic production deployment of generated projects, arbitrary repository modifications, multiple simultaneous users, shared cloud project storage, unrestricted project types, hosted open-source models and general database integration remain deferred. Generated plans also do not yet perform sourced competitor research or provide a dedicated ranked risk register.

Related Applications and Differentiation

We compared closely related tools using their primary documentation on November 1. These products already support significant planning, coding and orchestration. We do not claim that multiple agents or approval checkpoints are unique inventions [5-8].

Related application	Documented capability	ProjectPilot position and gap
OpenAI Codex [5]	Agent harness supports project context, tools, approvals and engineering workflows.	ProjectPilot offers a small browser workflow with explicit task contracts and dependency state. Codex provides substantially broader execution capabilities.
Claude Code [6]	Reads a codebase, edits files, runs commands and integrates with development tools.	ProjectPilot currently generates deliverable text without repository tools. Our intended advantage is a visible classroom workflow, not stronger code execution.
ChatDev [7]	ChatDev 2.0 supports configurable multi-agent workflows. Legacy ChatDev models a software company.	Our app narrows scope to one student and a small approved task graph. Multi-agent software generation already exists.
CrewAI [8]	Framework/platform documentation describes agents, tasks, crews and flows.	ProjectPilot supplies a particular student-facing workflow. CrewAI supports a much broader orchestration surface.

Differentiation to test

The proposed value is a guided sequence with editable requirements, explicit prerequisite outputs, individual approvals and focused revision history. Students can see what is waiting, what failed and what needs approval. Whether they prefer that experience over established tools remains an open user-research question.

Why use more than one role

Separate task contracts let a storage specialist define a schema before a dependent specialist uses it. The review role receives all outputs together and can flag disagreements. All roles currently use the same configured model with different instructions. Execution is serial. We have not shown that this arrangement produces better results or costs less than a single-agent baseline.

Five Biggest Risks

Risk exposure is estimated probability multiplied by extra team hours if the event occurs. These are planning judgments for the next increment, not measured frequencies. Higher exposure receives earlier mitigation.

Rank and risk	Probability	Impact	Exposure	Response and timing
1 Generated files do not build together	50%	16 h	8.0 h	Integration/verification role: choose one starter stack and run a minimal full build before adding features, milestone A.
2 Specialist interfaces disagree	35%	18 h	6.3 h	Specialist integration role: pass prerequisite outputs, define manifest/interface contracts, add cross-file integration checks in A and B.
3 API timeout or quota interrupts work	40%	10 h	4.0 h	Manager/API role: bounded retry, visible recovery and preserved outputs now. Measure run duration and add usage controls in B.
4 Runnable-project scope exceeds iteration	30%	12 h	3.6 h	Planning role: support a local single-user tracker first. Defer auth, external services and production deployment at scope review in A.
5 Students see little value over existing tools	40%	8 h	3.2 h	Research role: observe 3 to 5 students and compare a single-agent baseline before the scope expansion decision in C.

Existing controls and remaining limits

The current task graph rejects self, unknown and circular dependencies. Production API authentication blocks requests without an authorized session. Specialists receive prerequisite outputs. Revisions propagate to dependent tasks. A server call tries at most twice, and failures keep unrelated work. These controls reduce specific workflow risks. They do not prove that generated implementations compile or meet a user need.

The deployed prototype keeps the API key on the server. Live requests now require a private reviewer access code and explicit data-sharing consent. A signed HttpOnly session lasts eight hours, while Demo remains public. This is invite-based access, not individual user accounts or distributed per-user billing quotas. The project owner should control distribution of the access code and account budget.

Review points

At milestone A, stop expanding scope if the minimal manifest cannot assemble. At B, do not label a project verified if its build fails. At C, review observed student difficulties before claiming a durable reason to return to the product.

Architecture and Data Structures

React components present the workflow. ProjectStoreProvider holds the current project and schedules work. Browser localStorage saves the current single project under projectpilot-v2. Next.js routes call the OpenAI Responses API using server-only environment variables. The browser never receives the API key. Runtime validation checks the response independently of the requested JSON schema [9].

Structure	Key fields and responsibility
Project	id, title, description, targetUser, approvedSummary, status, createdAt. Owns the shared project brief.
ClarifyingQuestion	id, prompt, answer, why. Retains the original clarification context.
Requirement	id, projectId, text, priority, approvalStatus. Only approved requirements unlock task generation.
Task	id, projectId, title, description, ownerAgent, dependencies, expectedOutput, constraints, state, order. Defines the specialist contract and directed acyclic graph.
Agent and AgentOutput	Agent has role/capability/availability. Output has id, taskId, summary, assumptions, blockers, confidence, nextSteps, retryCount and approved.
Review	id, outputId, result, missingItems, conflicts, suggestions. One validated live review per output.
Feedback	id, projectId, userRequest, affectedTasks, revisionStatus. Open requests survive failures and inform retries.
WorkflowState	ProjectContext plus staleTaskIds, revisionTaskIds, skippedTaskIds, workflowNotice and optional draftBrief. Tracks invalidation and recovery without discarding history.
Persistence envelope	version 2, serialized state and mode. Uses one browser origin. It is not a shared database.
Metrics and ManagerNote	Session counts, user usefulness score and estimated time saved. Notes retain status explanations. Confidence is model-reported, not calibrated.

Organization

src/lib/types.ts models the domain. workflow.ts validates dependencies and approvals. store.tsx implements transitions. specialist-schema.ts validates outputs and reviews. specialist-client.ts sends browser requests. API routes own credentials and provider calls. Components separate early planning screens from execution, review and feedback. This domain separation keeps model calls out of rendering components.

Inputs Outputs and API Contracts

Input or event	Validation and output
Idea and target-user text	Start requires a brief of at least 12 characters in the UI. Produces a project and clarifying questions. Live mode needs server configuration.
Question answers	Every answer needs at least 12 characters. Produces pending requirements with priorities.
Requirement approval or edit	All requirements must be approved to generate tasks. Editing resets the edited approval and invalidates existing output context.
Task edits and plan approval	Title needs 3 characters and description 12, with a valid specialist, unique IDs and acyclic dependencies. Produces an approved executable task graph.
Specialist execution or manual retry	Approved plan/requirements and completed prerequisite outputs. Produces a pending AgentOutput or a visible failed task with retry.
Cross-output review	Requires all task outputs complete. Produces exactly one review per output or a visible retryable review failure.
Output approval or revision selection	Only completed current outputs can be approved. Missing reviews block final export. Revision selection identifies affected roots.
Feedback text	At least 12 characters and a selection. Live mode rewrites selected roots and transitive dependents. Unaffected approvals remain intact.
Refresh or reset	Refresh restores saved mode and state. Interrupted work becomes retryable. Confirmed reset clears the saved project.
Finalize and download	Approved requirements, current completed tasks, reviews and approved outputs are required. Output is Markdown containing requirements, contracts, deliverables, findings and revision history.

Server interfaces

GET /api/manager returns only a configured boolean. POST /api/manager accepts stage questions, requirements or tasks and project context. POST /api/specialist accepts stage execute or review, context, taskId for execution, optional feedback and previousOutput.

The manager body limit is 60,000 characters. The specialist body limit is 300,000 characters, and feedback has a 12,000-character limit. Unauthorized live requests return 401. Missing production access configuration fails closed with 503. Invalid JSON returns 400. Foreign origins return 403. Oversized requests return 413. Missing key returns 503. Provider/auth/quota/invalid-output failures return sanitized 502 errors. Specialist calls have up to two 24-second attempts within a 60-second route duration. These limits bound each call, not the total multi-task run.

Planning Use Cases

UC1 Create a project

Trigger and conditions: No active project and an available selected mode.

Normal flow and result: Enter the idea and audience, start clarification, and retain the original brief. Live mode requests project-specific questions.

Exceptions and recovery: Short input cannot start. An API failure preserves the input and exposes an error. Reset is needed to choose another mode after creation.

UC2 Answer clarification questions

Trigger and conditions: The manager has returned questions.

Normal flow and result: Enter all answers and generate pending requirements. Questions explain why an answer matters.

Exceptions and recovery: Blank answers block progression. Revisiting original answers after requirements does not overwrite later work. Refine scope through requirement editing.

UC3 Approve or update requirements

Trigger and conditions: Pending requirements are visible.

Normal flow and result: Review text and priority, edit if needed, approve each requirement, then generate or revisit tasks.

Exceptions and recovery: Pending/rejected requirements block task generation. Editing previously approved scope reopens planning, keeps task/output IDs and invalidates context-dependent outputs. Existing tasks require manual checking against the new scope.

UC4 Edit and approve task contracts

Trigger and conditions: Approved requirements and a task graph.

Normal flow and result: Edit title, description, specialist and dependencies. Inspect expected output, then approve the plan to start work.

Exceptions and recovery: Unknown IDs, duplicate IDs, self-dependencies and cycles fail validation. Editing does not run agents. Once the plan is approved, reopen scope through requirements before editing task contracts.

Consistency rules

Earlier screens remain viewable without regenerating data. Navigation cannot bypass approval gates or unlock final export. Requirements and task edits preserve history while invalidating affected work. Serial task scheduling is an implementation choice, not a requirement that users can never revise earlier decisions.

Execution Review and Recovery Use Cases

UC5 Run live specialists

Trigger and conditions: Approved requirements and an approved valid task graph.

Normal flow and result: Visit each task in topological order. Supply the role contract, approved scope and actual prerequisite deliverables. Validate the result, store it as pending human review and continue independent work.

Exceptions and recovery: An unmet prerequisite blocks a task. A model cannot assign its own approval or IDs. Invalid responses and timeouts receive one automatic retry. A persistent error marks the task failed and preserves unrelated outputs.

UC6 Retry or skip a failed task

Trigger and conditions: A failed task is visible on the board.

Normal flow and result: Retry only that task when prerequisites complete. Resume remaining ready tasks after success.

Exceptions and recovery: Duplicate run clicks are guarded. Skip marks the task skipped for now, not completed. Its dependents and final export remain blocked. Open feedback survives and is reused on retry.

UC7 Review outputs

Trigger and conditions: All task outputs complete.

Normal flow and result: The live review role reads outputs together against task contracts and approved requirements. Render per-output missing items, conflicts and suggestions.

Exceptions and recovery: Missing, duplicate or invented review output IDs fail validation. Review failure shows a message and Retry review, without rerunning completed workers. Finalization requires a review for each output. A pass is not proof of executable correctness.

UC8 Approve or request revision

Trigger and conditions: A current output and its findings are visible.

Normal flow and result: Approve the individual output or request a revision and continue to feedback. Findings remain visible.

Exceptions and recovery: Approving one output does not approve others. Stale, unfinished or unreviewed work cannot be finalized. A conflict blocks individual approval and final export until revision and review resolve it. Non-conflict missing-item flags remain visible for human assessment.

Implemented retry policy

The specialist route retries transient service failures, timeouts, incomplete responses and invalid structured results once. It returns authentication and rate/billing failures directly for configuration/account recovery. A user can then retry manually. There is no silent fallback from Live API to prepared demo data.

Revision Persistence and Export Use Cases

UC9 Apply user feedback

Trigger and conditions: One or more outputs selected, nonempty feedback, an approved current plan.

Normal flow and result: Compute all transitive dependents. Send feedback and the previous deliverable to each affected specialist in dependency order. Keep unrelated approvals and require fresh approval for regenerated work.

Exceptions and recovery: Failures retain old deliverables for reference, leave feedback open and route recovery through the board. After prerequisites succeed, resume blocked work. A requirement change can require manual task-contract updates as well as output revision.

UC10 Restore after refresh

Trigger and conditions: Browser storage contains a compatible saved project.

Normal flow and result: Restore project, mode, answers, outputs, approvals, feedback and current screen. Recover interrupted execution as retryable work.

Exceptions and recovery: Unavailable or malformed storage shows a notice and allows a usable new session. Browser storage is local to the origin. The localhost and Vercel domains do not share saves. Clearing browser data loses the saved session.

UC11 Finalize and export

Trigger and conditions: All current outputs are complete, reviewed and individually approved.

Normal flow and result: Finalize and download a Markdown package. It includes approved scope, dependency-ordered task contracts, deliverables, review findings and feedback history.

Exceptions and recovery: Missing approvals, stale outputs or unfinished prerequisites block finalization. Skipping feedback never approves pending work. Downloaded code is unexecuted text that still needs human integration and testing.

UC12 Reset the project

Trigger and conditions: No run in progress and an active project.

Normal flow and result: Confirm reset, clear the current browser save and start another project.

Exceptions and recovery: Cancel retains the saved project. Busy-state protection prevents reset from discarding a running operation.

State transitions

Project status progresses through draft, clarifying, requirements_pending, tasks_pending, agents_running, review_pending, feedback_pending/revising and complete. Task states include queued, blocked, in_progress, completed, needs_revision and failed. Revisions return to review and can repeat. Changed requirements return to planning. This makes the workflow revisable rather than a one-way waterfall.

Implementation and Test Evidence

Key comments explain the nontrivial controls: synchronous state snapshots guard rapid clicks, prerequisite outputs flow into role calls, dependent tasks invalidate after changes, and bounded server retries avoid an uncontrolled loop. IDs and approval state are application-owned. Model-generated summaries remain plain text in the UI and never execute as HTML or code.

Automated test	Result and important behavior
1 Input and requirements	Passed. Missing answers/approvals block progression. Editing preserves IDs.
2 Dependency edits	Passed. Rejects unknown/self/circular dependencies without executing work.
3 Backward navigation	Passed. Revisit does not overwrite work or bypass gates.
4 Targeted demo revision	Passed. Preserves unaffected approved outputs.
5 Failed task recovery	Passed. Isolates retry, duplicate clicks and skip behavior.
6 Changed approved requirements	Passed. Reopens plan and preserves history.
7 Export gates	Passed. Rejects unapproved state and labels estimates.
8 Live manager validation	Passed. Validates structured task plans and approval gate.
9 Origin validation	Passed. Accepts matching host and rejects foreign origin.
10 Manager API errors	Passed. Handles missing key, structured output and sanitized quota error.
11 Saved project recovery	Passed. Restores approvals/mode and interrupted execution.
12 Specialist input guards	Passed. Rejects unapproved/malformed/blocked/foreign requests.
13 Specialist service retry	Passed. One retry for invalid output, bounded timeout retry and no automatic auth/quota retry.
14 Review validation	Passed. Rejects missing/duplicate/invented review IDs and retains conflicts.
15 Live workflow with injected service	Passed. Prerequisite context, dependent revisions, preserved approvals, failure retry and missing-review gates.
16 to 21 Peer-review regressions	Passed. Keyword boundaries, conflict gates, signed access/consent, damaged saves, delayed timeout recovery and detailed export content.

Automated live-path tests use injected service responses. Separate real hosted checks passed specialist generation, review and revision. The hosted self-check returned reviewResult conflict, which demonstrates findings were surfaced rather than suppressed. A developer browser test used real UI actions for approvals, export, refresh and mobile layout. Its Live-mode revision UI portion used a deterministic mock service, separately from the real API test [2].

npm run build passed TypeScript and production compilation. ESLint reported zero errors and zero warnings after removing the unused capture helper. Passing these checks does not establish generated-project build correctness.

Setup Run and Verification Instructions

Open the hosted application

Visit https://projectpilot-ai-cse3311.vercel.app. Select Demo for prepared samples or Live API for real manager, specialist and review calls before starting a project. Live mode requires a private code from the project owner and consent to sharing the listed context with OpenAI. Do not put the reviewer code in the public submission. Live calls use the server account. A browser retains one current project. Demo needs no signup. Live access uses an invitation code, not an individual account.

Run from source

Prerequisites: Node.js 22 and npm. Obtain the November 1 implementation files, including the new specialist route/client/schema, rather than assuming the older repository HEAD contains them. From the repository root run:

```
npm ci
cp .env.example .env.local
# Set OPENAI_API_KEY in .env.local
# Optional OPENAI_MODEL defaults to gpt-4o-mini
# Production also requires LIVE_ACCESS_CODE
npm run dev -- --port 3313
```

Open http://localhost:3313. Restart the server after changing environment variables. Never commit .env.local or put an API key in a NEXT_PUBLIC variable. The app has no separate native compilation step. npm run build produces the production Next.js build and npm run start serves it.

Run automated checks

```
npm test
npm run lint
npm run build
```

Optional browser and live provider checks

The optional browser scripts require Playwright, Google Chrome and the bundled Playwright WebKit browser for the peer-review suite. In the development environment Playwright was already installed. For a fresh optional tool setup run npm install --no-save playwright, then npx playwright install webkit. Video recording also needs npx playwright install ffmpeg. This setup is not required to run the application or Node workflow tests.

```
PROJECTPILOT_URL=http://localhost:3313 node scripts/verify-iteration3-browser.mjs
PROJECTPILOT_URL=http://localhost:3313 node scripts/verify-iteration3-live.mjs
PROJECTPILOT_URL=http://localhost:3313 node scripts/verify-peer-review.mjs
```

The live verification script makes three billable API calls. It checks a ProjectPilot retry-design example and requires a running server with a configured key. The browser smoke uses a fresh isolated context, so it does not reset a normal browser save. The submitted demo uses the public Vercel app and real model responses. Captions identify the steps. It is a software demonstration, not a student usability study.

Troubleshooting

503: configure the key and restart or redeploy. 502: read the sanitized message, correct the account/model setting if needed, then retry. Failed task: finish prerequisites, Retry task, then run remaining ready tasks. Missing review: Retry review. Saved-state issue: download completed work when possible before resetting. No live response is silently replaced by sample output.

Screen Evidence and Planned Project Generation

The current execution and feedback screens support this iteration. The accompanying demo shows actual hosted API calls. Current screenshots show the product as it runs, rather than a claim that future file assembly or test execution already exists.

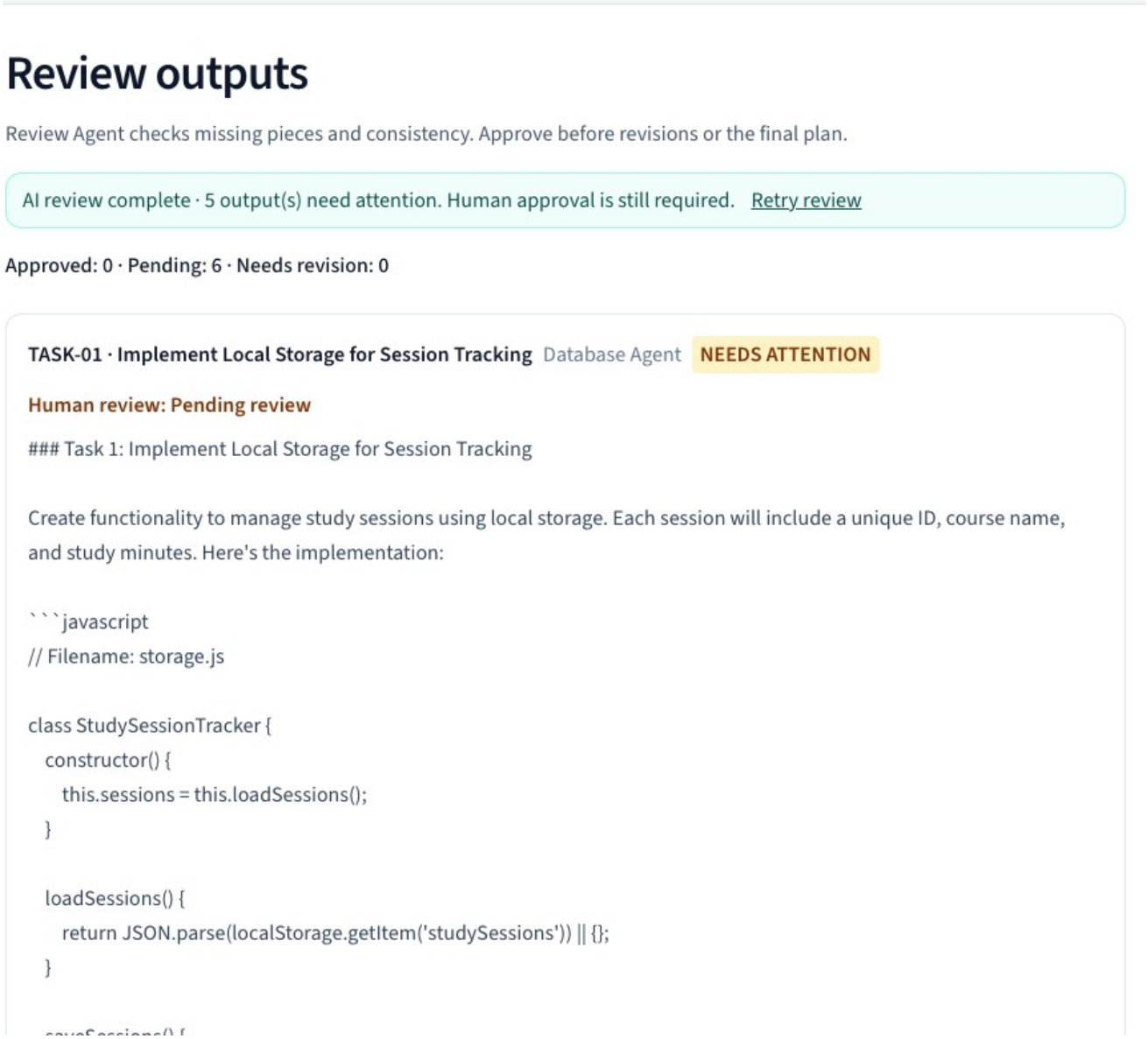


Figure 1 Current live output review detail

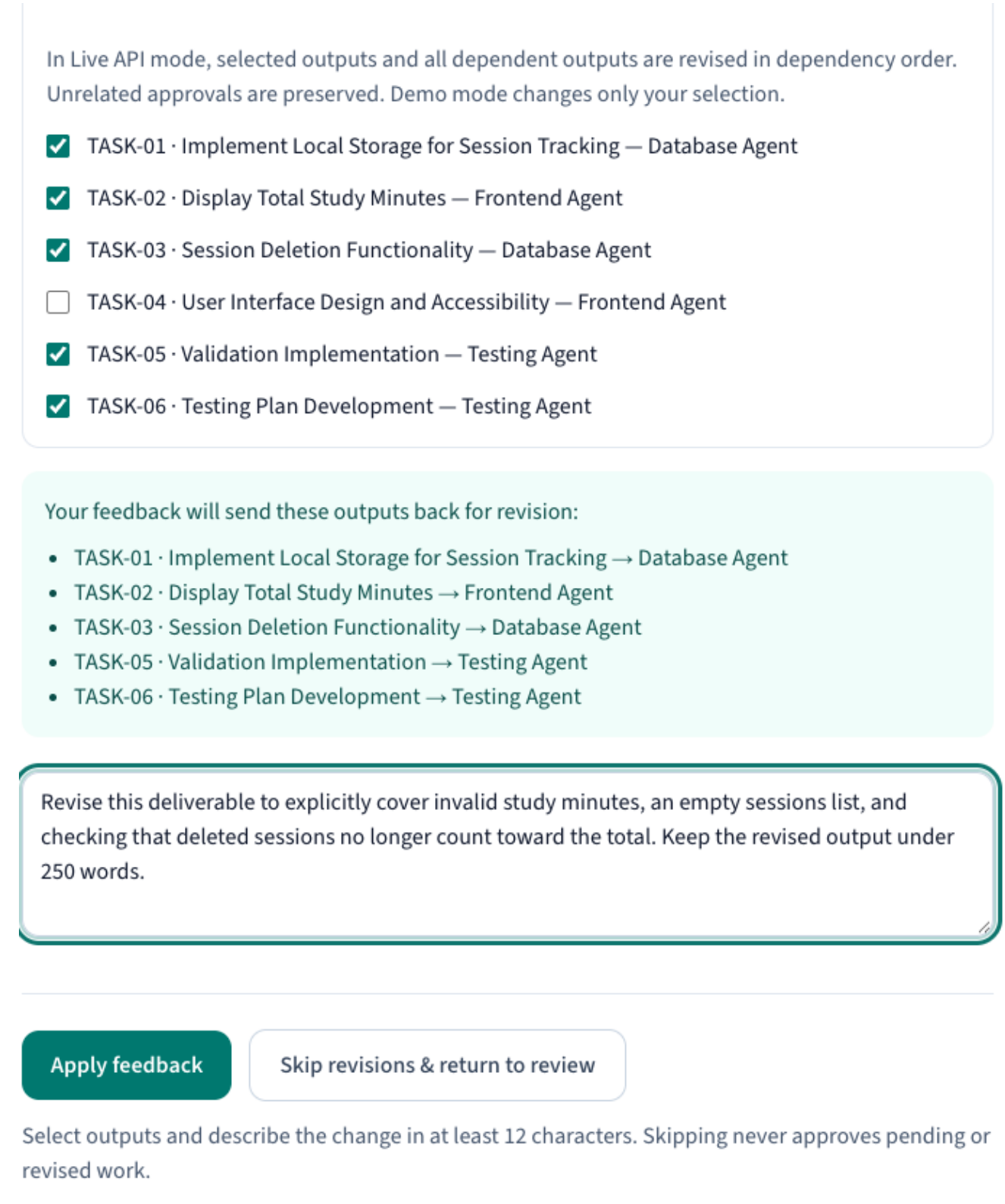


Figure 2 Current feedback screen detail

Planned Screens for the Next Iteration

Project files: filename tree, selected editable file, generation state and Download ZIP. Build results: command, status, duration, log output and bounded repair action. Preview: generated app, supported runtime and Restart preview. Feedback: requested change, affected file list, revision summary and verification results. These are proposed screen contents, not implemented screenshots.

Proposed screen	Main contents	Transitions and exceptional paths
Project files	File tree, selected file content, generated state and Download ZIP.	Select a file to inspect. Missing/unsafe paths prevent packaging and return to generation.
Build and test results	Command, exit status, log output and repair attempts.	Success opens preview. Failure preserves logs and offers a bounded repair attempt.
Working preview	Generated app, runtime status and restart control.	A stopped or crashed preview shows an error and returns to verification.
Change request	Feedback, affected files, revision summary and new results.	User approval starts file revision. Failed verification keeps the previous accepted project available.

Proposed file contract

The first supported generator should return a bounded file manifest containing path, language, complete content, task ID and prerequisite contracts. The assembler should reject unsafe or duplicate paths, define one dependency manifest and keep generated test results separate from model claims. This contract is proposed next-iteration work.

Proposed acceptance example

For the study tracker, a student should be able to download a package, install dependencies using its README, start the application, add a valid session, reject invalid minutes, remove a session and see the total update. The system should report the actual commands and results from isolated verification. Iteration 3 does not yet satisfy this complete-project example.

References and Submission Notes

[1] Team 5. ProjectPilot AI deployed application. https://projectpilot-ai-cse3311.vercel.app. Source: https://github.com/manit0700/projectpilot-ai-cse3311. November 1, 2026 working-tree deployment.

[2] Team 5 developer evidence. tests/workflow.test.cjs; scripts/verify-iteration3-browser.mjs; scripts/verify-iteration3-live.mjs; docs/ITERATION_3.md. Local and hosted verification performed November 1, 2026. Peer-review browser results: docs/iteration3-evidence/peer-review-browser-results.json. Companion demo evidence is retained with the submission build materials.

[3] Christoph Csallner. Iteration 2 feedback to Team 5, October 26, 2026 at 3:27 p.m. Course comments supplied by the team on November 1. Comments include specialist retry, feedback, font size, examples, scope, related tools and testing.

[4] Team 6 inception review of Team 5. Retained project notes TEAM6_REVIEW_ACTION_ITEMS.md and NEXT_SUBMISSION_RECOVERY_PLAN.md. Historical peer evaluation, not a new external user interview.

[5] OpenAI Developers. Codex as a platform: build on the open agent harness. https://developers.openai.com/blog/codex-as-a-platform. Accessed November 1, 2026.

[6] Anthropic. Claude Code overview. https://code.claude.com/docs/en/overview. Accessed November 1, 2026.

[7] OpenBMB. ChatDev project documentation. https://github.com/OpenBMB/ChatDev. Accessed November 1, 2026.

[8] CrewAI. Introduction. https://docs.crewai.com/en/introduction. Accessed November 1, 2026.

[9] OpenAI. Structured model outputs. https://developers.openai.com/api/docs/guides/structured-outputs. Accessed November 1, 2026.

[10] Vercel. Environment variables CLI documentation. https://vercel.com/docs/cli/env. Next.js route-handler reference supplied with the installed package: node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/route.md.

[11] Team 6. Iteration 2 Review of Team 5. Eighteen numbered findings supplied by the team as a screenshot and text on November 1, 2026. Reviewers David, Ujjwal Sood and Hardik Kari. Disposition tables preserve the distinction between implemented work and remaining issues.

Third party software and visual material

Project dependencies include Next.js, React, TypeScript, Tailwind CSS and ESLint, recorded in package.json and package-lock.json. Integration uses OpenAI Responses and Vercel hosting [9-10]. All product screenshots and demo footage depict Team 5's application. AI-generated deliverable text in the demo is labeled by its Live API context and does not represent human-authored or executed application code. No stock imagery is used. OpenAI Codex assisted implementation and preparation of these materials; the team remains responsible for reviewing and presenting them.

Remaining evidence and release work

External student feedback, a measured comparison with one agent, verified runnable generated projects and named team ownership remain outstanding. The implementation is committed and pushed to GitHub. The next iteration focuses on file assembly, actual build/test execution and a runnable preview. Risk estimates and proposed dates should be revisited when the team assigns owners.